

Projektbericht

Zum Projekt **me@hsrcm**

Studienleistung im Rahmen des Moduls
„Portfolio Medieninformatik“

Student: David Thomann

Zeitraum: 01. November 2025 – 28. Februar 2026

Fachbereich: Design Informatik Medien

Studiengang: Medieninformatik B.Sc.

Betreuer: Prof. Dr. Jörg Berdux

Stand: 3. April 2026

Inhaltsverzeichnis

1	Einführung	2
2	Grundlagen	2
2.1	Problem- und Kontextanalyse	2
2.2	Zielgruppenanalyse	3
2.3	Bestehende Lösungen und Marktüberblick	3
2.4	Fachliche Anforderungen	3
2.5	Rahmenbedingungen & Einschränkungen	3
3	Konzept	4
3.1	Nutzerszenarien	4
3.2	Funktionale Anforderungen	5
3.3	Styleboard	6
3.4	Screens	6
3.5	Logo	7
4	Architektur	8
4.1	Überblick	8
4.2	Domäne	9
4.3	Schnittstellen	11
4.4	Komponenten	12
5	Umsetzung	13
5.1	Verwendete Technologien	13
5.2	Implementierung von Kernfunktionen	14
5.3	Tests und Qualitätssicherung	15
6	Fazit	17
6.1	Zielerreichung und Ergebnisbewertung	17
6.2	Ausblick und Weiterentwicklung	17
	Literatur	19
	Glossar	19

1 Einführung

Im Rahmen des Studienprogramms des B.Sc. Medieninformatik an der Hochschule RheinMain[1] wird im letzten Semester vor der Bachelorarbeit ein Modul **Wahlmodul Portfolio Medieninformatik** angeboten. Als Studeinleistung dieses Moduls könnten Studierende entweder weitere Wahlmodule aus den Wahlmodulkatalogen belegen oder alternativ ein Projekt in einem für sie interessanten Bereich der Medieninformatik entwickeln. Insgesamt muss so aus einer Kombination und Projekten und Wahlmodulen ein Arbeitsumfang von 10 Credit Points (≈ 360 Arbeitsstunden) entstehen.

Ich entschied mich für ein Einzelprojekt mit einem Arbeitsaufwand der vollen 10 CP im Bereich mobiler, mehrplattformiger Anwendungen. Ein klassischer Projekt in dem man viele experimentieren kann und zu dem es viele Ressourcen gibt ist das einer Campus App. Je nach Ausrichtung kann man hier mit vielen externen Schnittstellen arbeiten, aber auch die gerätespezifischen Informationen (bspw. GPS bei Smartphones) nutzen.

2 Grundlagen

Im Entwicklungsprozess einer solchen Projekts gilt es zunächst Anforderungen und Umfeld zu analysieren. Dies hilft den späteren Rahmen der Anwendung zu konkretisieren und bereits bestehenden Anknüpfungspunkte zu finden.

2.1 Problem- und Kontextanalyse

Im Hochschulalltag nutzen Studierende verschiedene Plattformen um untere anderem auf Lerninhalte zuzugreifen, sich über Vorgänge an der Hochschule zu informieren und organisatorische Prozesse abzuwickeln. Je nach Hochschule sind all diese Plattformen mehr oder weniger verstreut und unterscheiden sich teilweise auch auf Studiengangsebene, wenn nicht sogar auf Lehrveranstaltungsebene. Gerade neuen Studierenden fällt es so schwer, sich im Alltag zu orientieren, was zu Reibungsverlusten allein durch die Komplexität der Informationsquellen führen kann. Eine Vereinfachung und Vereinheitlichung hochschulseitiger Schnittstellen ist deswegen extrem sinnvoll und sollte auf möglichst vielen Plattformen passieren (Mobil, Web, Desktop/Laptop), wobei mobile Anwendungen sicherlich am besten in den dynamische Studienalltag passen. Aus diesem Grund setzen viele Hochschulen auf eine Applikation, die unterschiedlich viele Dienste und Anwendungen vereint oder zumindest Schnittstellen zu anderen Anwendungen bereitstellt. Im Rahmen meines Projekts me@hsrm habe ich eine solche (mobile) Anwendung entwickelt und dabei einige der zig User Cases für solch eine Campus App umgesetzt.



2.2 Zielgruppenanalyse

Die Zielgruppe meines Projekts waren Studierende, mit einem leichten Schwerpunkt auf Studierende in den ersten Semestern. Diese haben ein Interesse daran, sich an ihrem Campus gut zurechtzufinden, sowie ihren Studienalltag rund um den Campus einfach zu bewältigen.

2.3 Bestehende Lösungen und Marktüberblick

Der größte Anbieter von Campus Apps im deutschsprachigen Raum ist Uninow[2]. Dieser bietet verschiedene Lösungen für Probleme aus dem Hochschulalltag an. Dazu gehören unter anderem Buchungs- und Ausweissysteme aber auch Sicherheitsanwendungen um hochschulinterne Inhalte zu schützen. Das größte Produkt hingegen ist eine standardisierte App für Hochschulen, die bei Interesse auf eine bestimmte Hochschule spezialisiert werden und um viele Features erweitert werden kann.

Trotz solcher gut etablierten Anbieter gibt es aber auch Hochschulen, die eigene Apps entwickeln. Hierzu gehört beispielsweise die Universität Augsburg, die mit ihrer Campus App ihren Studierenden eine Multiplattform-Anwendung mit vielen Features zur Verfügung stellt. Hierzu gehören Campus-Navigation, ein Stundenplan, Informationen über dein Speiseplan der Mensa und mehr. [3]

2.4 Fachliche Anforderungen

Ziel ist es eine Anwendung zu schaffen, die Studierenden in möglichst vielen Bereichen ihres Hochschulalltags hilft und sie unterstützt. Da sicherlich nicht alle Bereiche abgedeckt werden können ist es wichtig, sich auf die zentralsten und alltäglichsten Problemstellungen zu fokussieren. Dabei sollte die Nutzbarkeit auf verschiedenen Plattformen gewährleistet sein und ein einheitliches Anwendungserlebnis vermittelt werden. Die Informationen in der Anwendung sollten möglichst aktuell und sauber aufbereitet werden und intuitiv einsehbar sein.

2.5 Rahmenbedingungen & Einschränkungen

Das Projekt wird als Einzelarbeit mit kurzem zeitlichen Rahmen absolviert. Der Funktionsumfang sollte dementsprechend eher gering gehalten werden, damit alle Funktionen sauber umgesetzt werden können. Da es sich um eine studentische Arbeit handelt, ist eine Anbindung an echte Hochschulsysteme ausgeschlossen. Funktionalitäten in diese Richtung müssen also mehr oder weniger detailliert simuliert werden. Sonstige Datenquellen sollten möglichst öffentlich sein, damit eine theoretische, echt Implementierung an einer Hochschule möglich bleibt. Es sollte des weiteren darauf geachtet werden, möglichst zukunftsfähige Technologien einzusetzen.



3 Konzept

3.1 Nutzerszenarien

Szenario 1 – Pendeln

Wer? Ein Student der weiter weg von seinem Campus wohnt und somit täglich von und zum Campus pendelt.

Ziel: Der Student möchte eine möglichst stressfreie und verlässliche Reise zum Campus finden und wissen. Außerdem möchte er sich einfach auf die Verhältnisse vor Ort einstellen können.

Handlungen: Der Student lässt sich morgens eine Verbindung anzeigen, die ihn pünktlich zum Campus bringt. Dabei schaut er ebenfalls auf die aktuellen Wetterverhältnisse um sich gegebenenfalls einen Regenschirm oder einen Schal mitzunehmen. Nach einem Tag mit Lehrveranstaltungen sucht er nach den nächsten Verbindungen von einer der Haltestellen am Campus um einzuschätzen, ob er sich beeilen muss oder ob er seinen Bus pünktlich erreicht.

Szenario 2 – Das erste Semester

Wer? Eine Studentin im ersten Semester, die neu in der Stadt ist und sich auf dem weitläufigen Campus noch nicht zurechtfindet.

Ziel: Sie möchte sich schnell orientieren, und Räume in denen ihre Veranstaltungen stattfinden schnell finden. Auch über Gebäude in ihrer Nähe möchte sie sich informieren können.

Handlungen: Die Studentin nutzt die interaktive 3D-Campus-Karte, um sich über Gebäude um sie herum zu informieren und lässt sich automatisch das Gebäude in dem ihre nächste Lehrveranstaltung stattfindet optisch hervorheben.

Szenario 3 – Mittagspause

Wer? Eine Gruppe von Studierenden, die in der Mittagspause in der campuseigenen Mensa essen möchte und bereits auf dem Weg dorthin das Speiseangebot betrachten möchte.

Ziel: Die Studenten der Gruppe möchten ihre Entscheidung, welches Gericht zu nehmen wollen, möglichst frühzeitig treffen, da es zur Mittagszeit in der Mensa schnell voll werden kann.

Handlungen: Die Studenten rufen die Speiseplan-Ansicht der Anwendung aus und rufen ihre Mensa auf. Dort klappen sie den Eintrag für den aktuellen Tag auf und schauen sich die Gerichte und Beilagen an. Anhand des Preises und der Beschreibung treffen sie ihre Entscheidung.



Szenario 4 – Veranstaltungsalldag

Wer? Ein Student in einem höheren Semester, der verschiedenste Veranstaltungen aus dem Hochschulangebot belegt hat und mehrere Klausuren schreibt. Er ist etwas unorganisiert, pflegt keinen Kalender und schaut selten in sein E-Mail-Postfach

Ziel: Der Student möchte seine Termine an der Hochschule einfach organisiert bekommen und nachschauen können, welche Termine er an einem beliebigen Tag hat. Zu Klausuren möchte er erinnert werden, sich anzumelden.

Handlungen: Der Student ruft seinen Stundenplan auf und sieht seine Seminar und Vorlesungen. In der Prüfungszeit sieht er zusätzlich Einträge für Prüfungen. Wenn er die Anwendung aufruft wird er automatisch auf nicht angemeldete Prüfungen in Modulen, die er belegt hat hingewiesen.

3.2 Funktionale Anforderungen

Aus diesen Nutzerszenarien ergaben sich folgende funktionalen Anforderungen und ihre Gruppierung in Modulen:

Kartenmodul Ermöglicht die Anzeige von Gebäude-Modellen, die auf der Karte hervorgehoben werden und das Anklicken diese Modelle um mehr Informationen zu erlangen

Mensamodul Ermöglicht dem Nutzer Speisepläne seiner Hochschulmensen anzuzeigen inkl. des Preises und aller Informationen zu Hauptgängen und Beilagen

Stundenplan und Prüfungsmodul Zeigt dem Nutzer einen Kalender mit seinen belegten Veranstaltungen und Prüfungen an und ermöglicht die einfache Navigation zu den Veranstaltungsorten über das Kartenmodul

Verkehrsmodul Sucht dem Nutzer automatisiert Verbindungen seiner Heimathaltestelle zu seiner nächsten Veranstaltung und ermöglicht ihm leicht nach den nächsten Abfahrten an einer Haltestelle in der Nähe des Campus zu suchen

Wettermodul Gibt dem Nutzer automatisch Informationen über die Wetterverhältnisse an einem Campus

3.2.1 Hierarchie

Den Modulen, mit Ausnahme des Wettermoduls wurden eigene Seiten gegeben, da sie sich fachlich recht stark unterschieden. Zusätzlich wurde eine Seite für Einstellungen und eine Homepage erstellt, auf dieser werden die wichtigsten Informationen verschiedener Module zusammengefasst.

So ergab sich eine flache Hierarchie, in der alle Funktionalitäten über maximal zwei Aktionen aufgerufen werden können:





Abbildung 1: Flache Hierarchie der Seiten

3.3 Styleboard

Für das Projekt wurde ein Styleboard erstellt, dass auf die wichtigsten Punkte der Gestaltung eingeht um eine grobes Gefühl für das Auftreten der Anwendung zu erhalten. Dabei wurde auf abgerundete Formen und wenige markante Farben geachtet. Später wurde dieses Styleboard mit dem Material Design Builder erweitert, weswegen die Abbildung hier nicht der endgültigen Version entspricht.

Farben



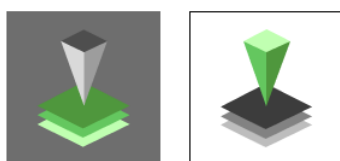
Schrift

Willkommen am Campus
Unter den Eichen

Dies ist ein weniger
weniger wichtiger Text

und hier wird es nochmal
unwichtiger

Logo



Formen



Abbildung 2: Styleboard vor der Implementierung

3.4 Screens

Da auf Wireframes verzichtet wurde fand die Entwicklung der Screens direkt in der App statt. Es wurden zwei Farbthemes umgesetzt um auf Nutzerpräferenzen bezüglich Helligkeit einzugehen:



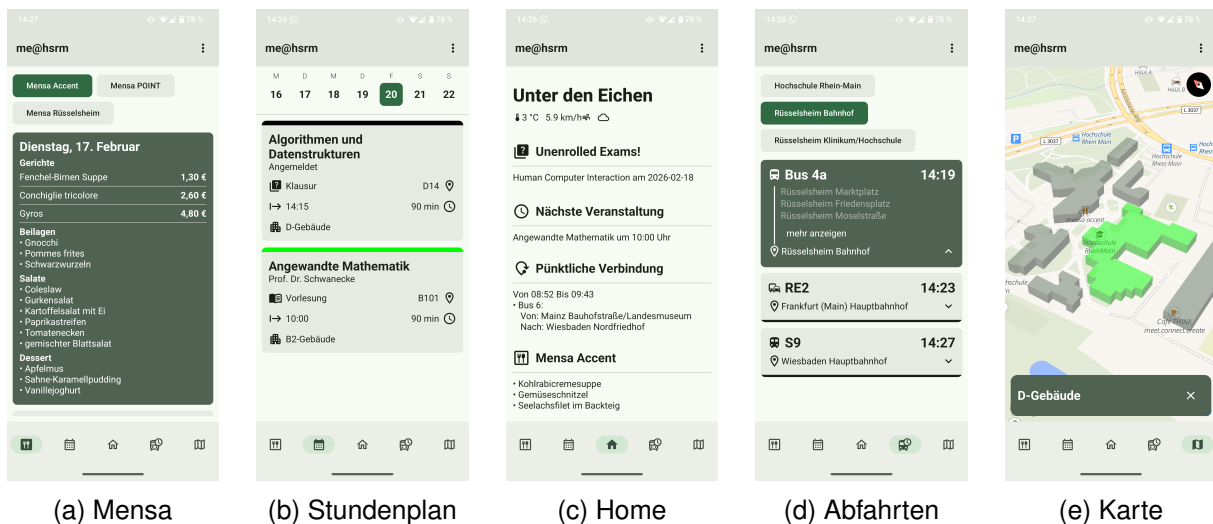


Abbildung 3: Die fünf Hauptscreens im Light Mode

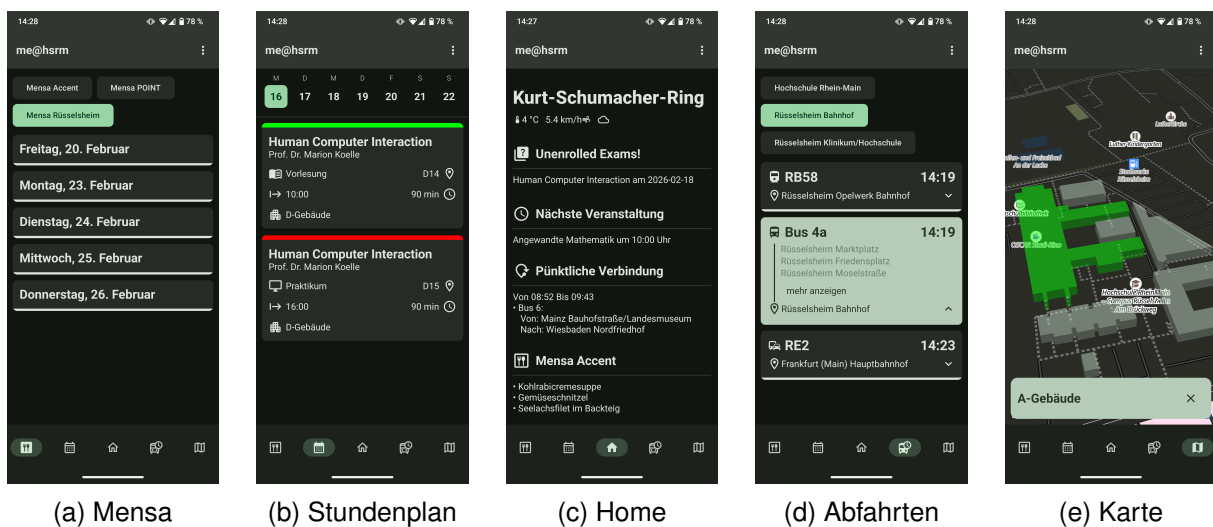
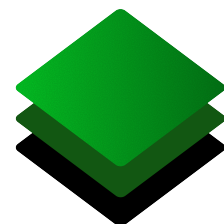


Abbildung 4: Die fünf Hauptscreens im Dark Mode

3.5 Logo

In 2 gab es bereits einen Kandidaten für ein Logo. Auf diesem sollte die eigentlich wichtigste Funktion zur Orientierung auf dem Campus, der Karte erkennbar sein. Der Pin, zur Markierung einer Position, wurde aber bei kleineren Versionen des Logos schlecht erkennbar, außerdem war die Formsprache sehr kantig. Letztendlich wurde ein neues, etwas minimalistischeres Logo erstellt:



4 Architektur

4.1 Überblick

Für meine Anwendung habe ich eine klassische Schichtenarchitektur, die Verantwortlichkeiten kapselt ausgewählt. Diese sind organisiert nach der Nähe zu den eigentlichen Daten und der Anwendungslogik beziehungsweise der Nähe zum Nutzer. Folgende Schichten existieren:

Präsentationsschicht Besteht aus der Benutzeroberfläche für die verschiedenen Plattformen.

Präsentationslogikschicht Speichert den Zustand der Präsentationsschicht und stellt die Operation für Nutzereingaben bereit.

Datenzugriffsschicht Kapselt die Datenbeschaffung aus möglichen externen Quellen oder persistenten lokalen Quellen.

Domänen-/Modellschicht Diese Schicht besteht aus dem Modell – Also dem Teil des Hochschulumfelds, dass für die Applikation relevant ist und abgebildet werden soll.

Datenschicht Speicherung und Besorgen von Modelldaten und Nutzereinstellungen der App.

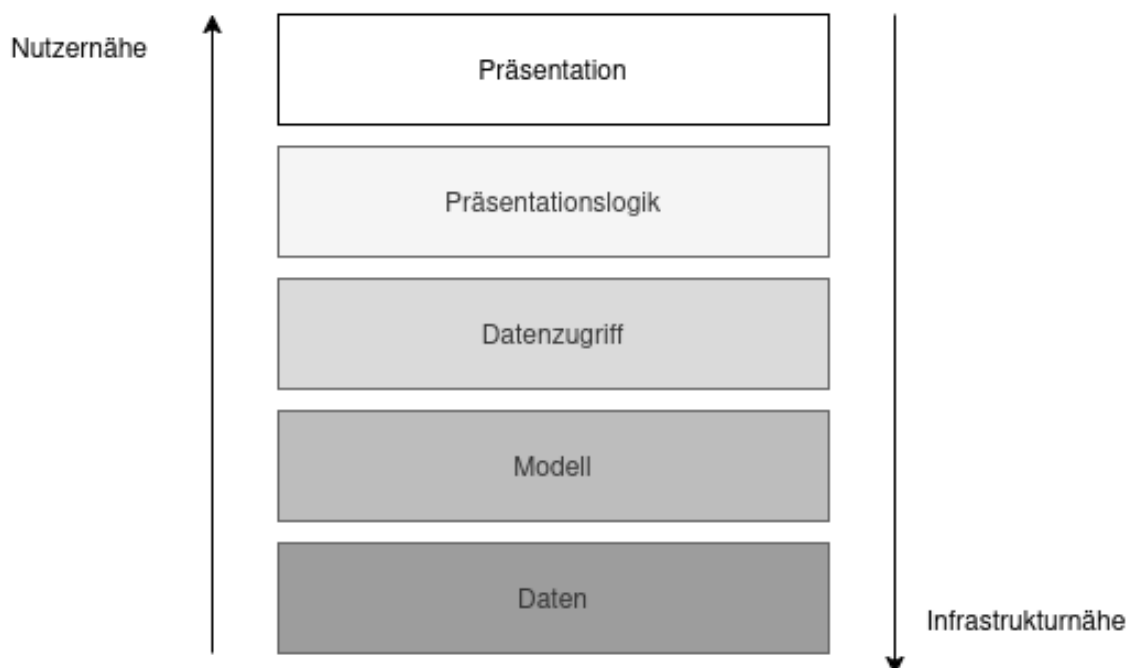


Abbildung 5: Architektur als Schichtenmodell



4.2 Domäne

Den fachlichen Einstiegspunkt bildet die Entität des Campus. Ihr sind Haltestellen (Stops) mit Abfahrten (Departure) und Reisen (Journey) zugeordnet, sowie Mensen (Canteen) in denen Speisepläne (Menus) mit Speisen (Dishes) und Beilagen (Sidedishes) angeboten werden. Außerdem gehören zu einem Campus Gebäude (Buildings) in denen Kurse (Courses) gehalten werden. Einige UI-spezifische Details (bspw. Anzeigefarbe eines Kurstyps) sind der Einfachheit halber in der Domäne mitabgebildet aber nicht verpflichtend, sodass eine losere Kopplung besteht.

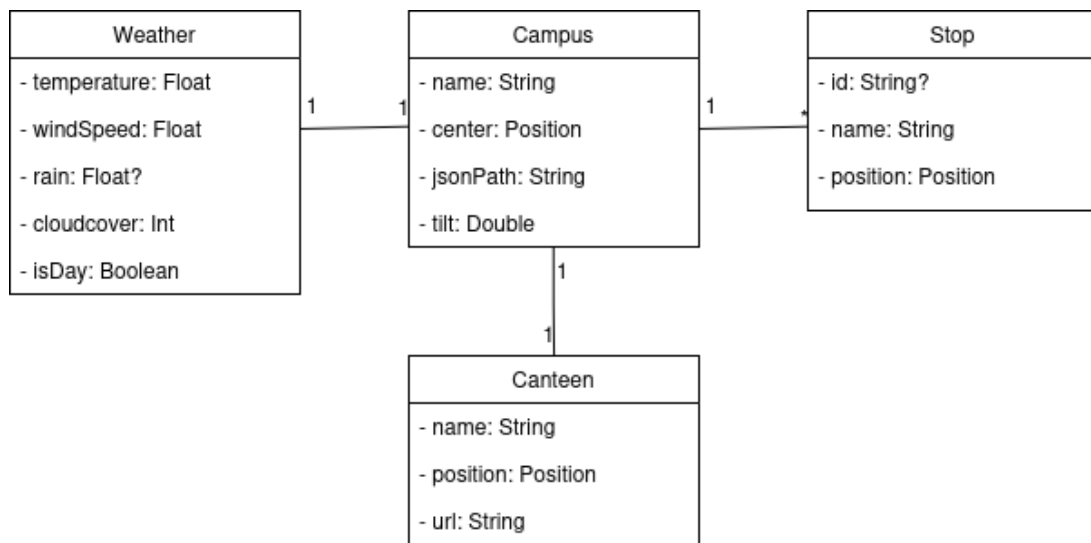


Abbildung 6: Fachdomäne rund um den Campus

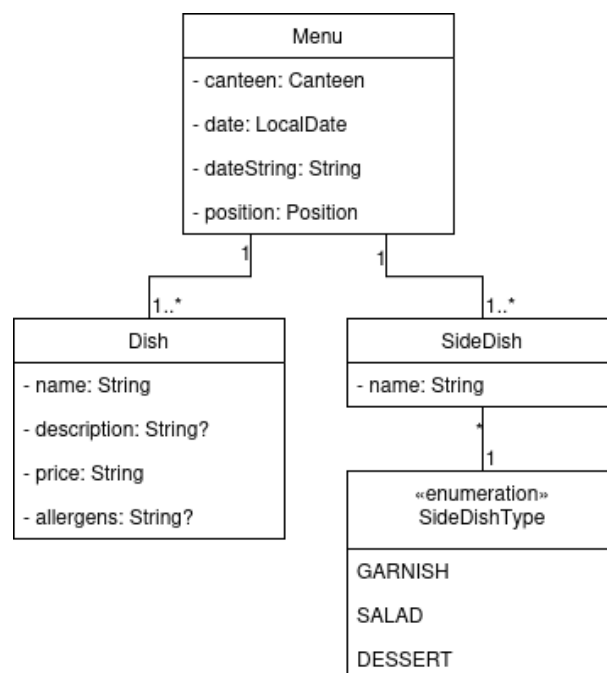


Abbildung 7: Fachdomäne rund um den Speiseplan



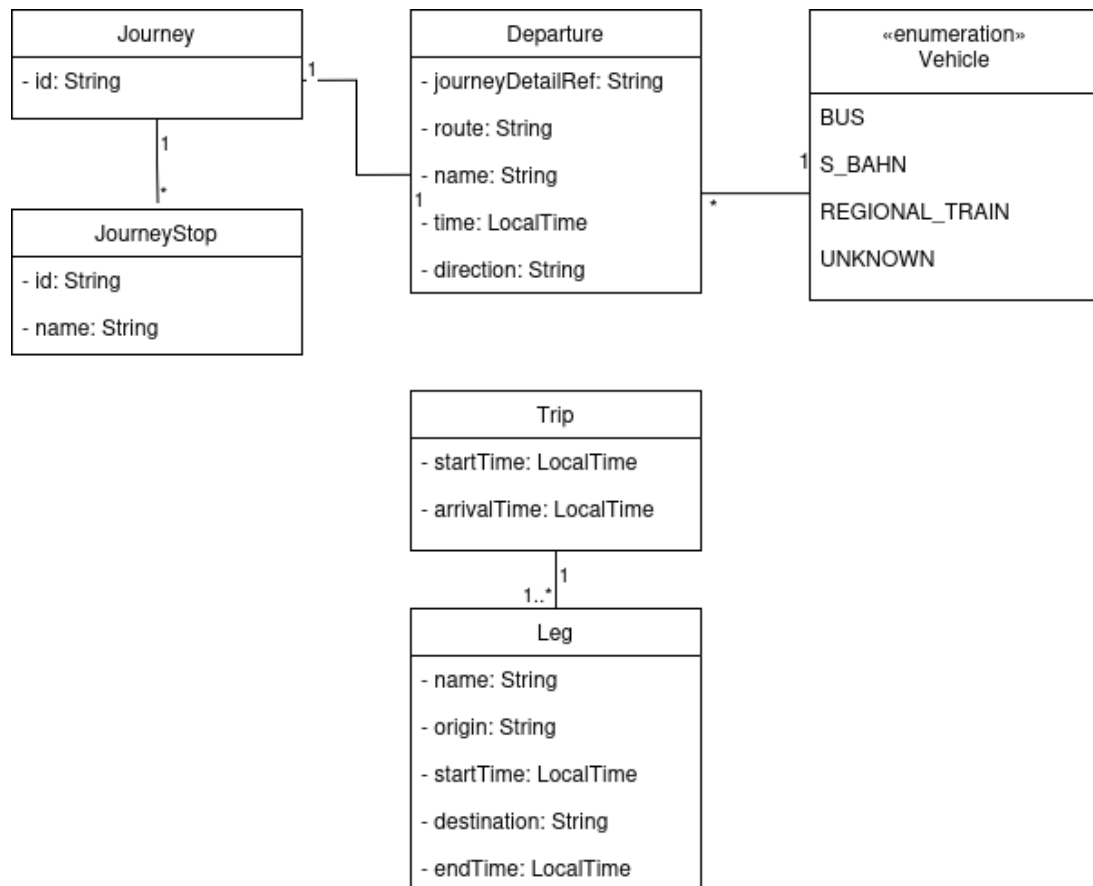


Abbildung 8: Fachdomäne rund um Fahrten

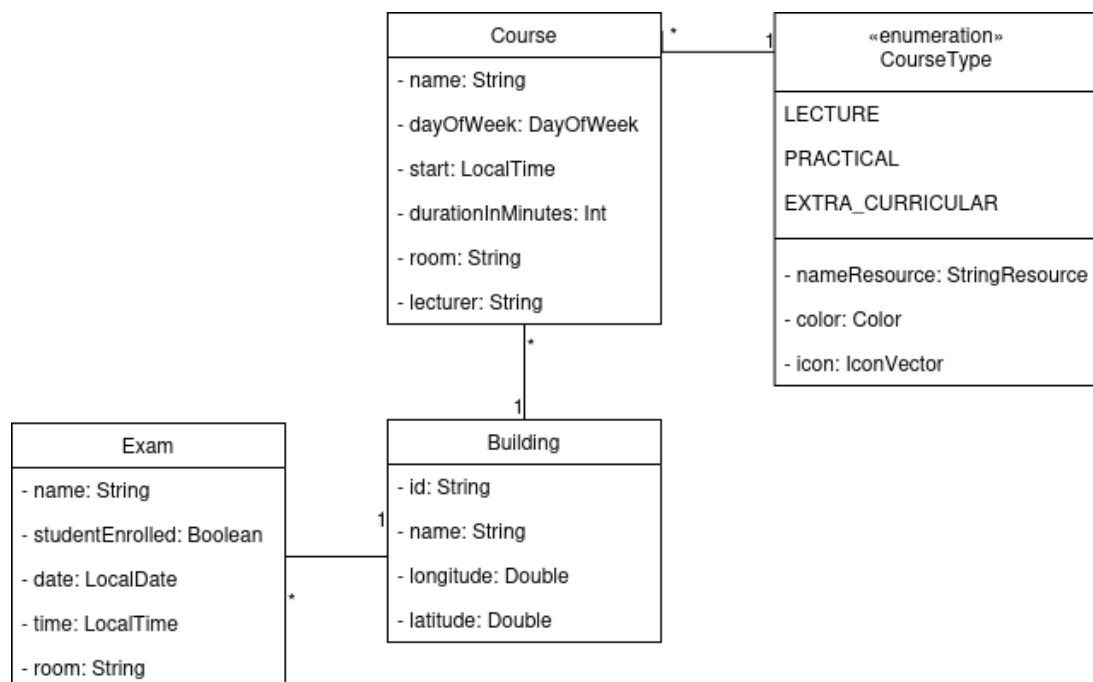


Abbildung 9: Fachdomäne rund um Kurse und Klausuren



4.3 Schnittstellen

Für die Beschaffung von Daten des Öffentlichen Nahverkehrs, den Karten- sowie Wetterdaten und den Speiseplänen der Mensa mussten externe Schnittstellen angebunden werden. Diese unterscheiden sich in ihrer Anbindung und dem Aufbau der Daten.

Nahverkehr

Der Rhein-Main-Verkehrsverbund stellt unter RMV Open Data [4] Infrastrukturdaten und Informationen zum Fahrplan zur Integration in eigene Anwendungen zur Verfügung. Nach der Beantragung eines API-Schlüssels kann man dann Livedaten beispielsweise zu Abfahrten an einer bestimmten Haltestelle, Fahrtverläufen einer bestimmten Fahrt oder Haltestellen mit bestimmten Namen erhalten. Die API besitzt eine SwaggerUI-Dokumentation was die Implementierung über einen eigenen Client sehr leicht macht. Sie liefert zudem Daten in verschiedenen Formaten (XML und JSON).

Geodaten

Für die Daten zu Gebäudegrundrissen habe ich die OpenStreetMap [5] genutzt. Die Daten kann man vereinfacht abfragen über Anbieter wie Overpass Turbo [6] Hierüber konnte ich GeoJSON-Daten für die Gebäude an den verschiedenen Campussen abfragen und diese dann in MapLibre laden.

Mensadaten

Leider wird für die Mensen des Studierendenwerk Frankfurt [8] keine strukturierte API zur Verfügung gestellt, weswegen die Speise und Beilagendaten aus den öffentlich zugänglichen Speiseplänen extrahiert wurden für die verschiedenen Standort extrahiert werden mussten.

Wetterdaten

Open Meteo [7] stellt eine schlüssel- und zugriffsratenfreie API für verschiedene Wetterdaten zur Verfügung. Ortsbasiert können Informationen zu stündlichen, täglichen und aktuellen Wetterverhältnissen abgefragt werden. Relevant für das Projekt waren hier vorallem die aktuellen Daten zu Temperatur, Regenfall, Windgeschwindigkeit und Bewölkung.



4.4 Komponenten

Beispielhaft und vereinfacht sollen hier die am Speiseplanmodul beteiligten Komponenten gezeigt werden:

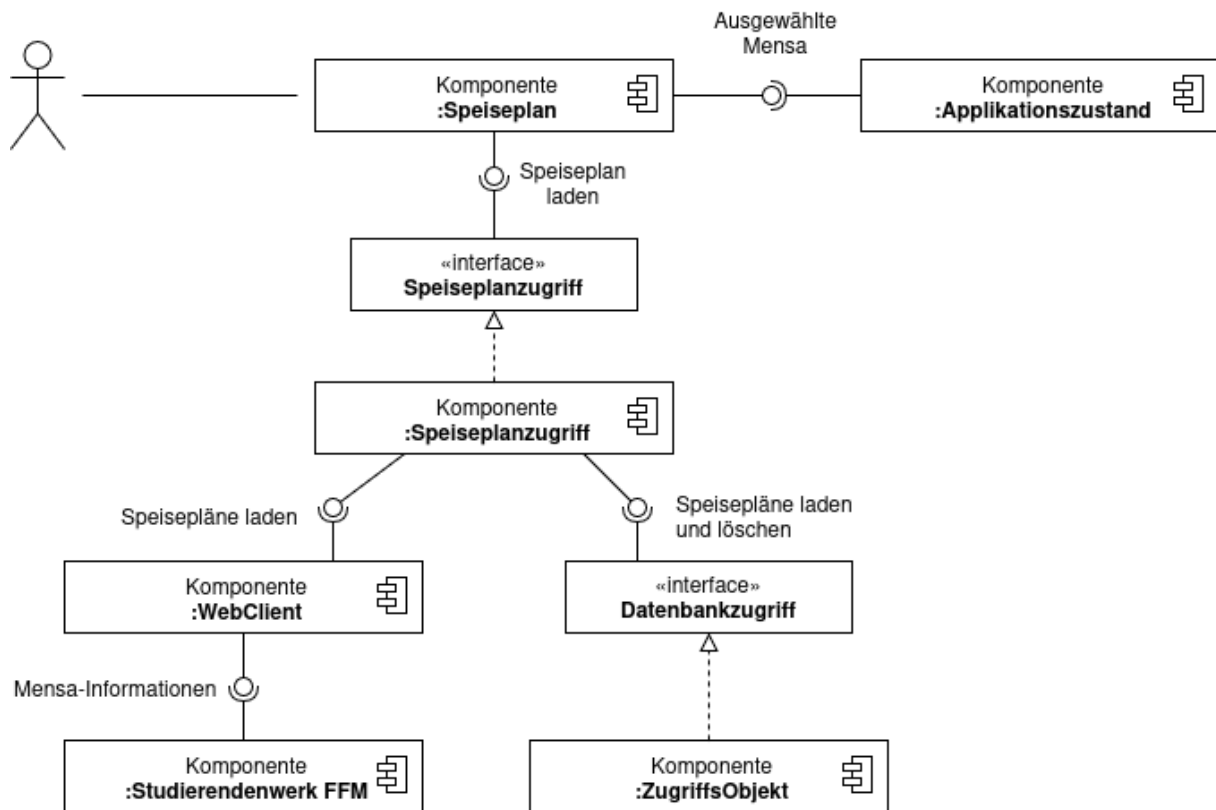


Abbildung 10: Komponentendiagramm des Speiseplan-Moduls

5 Umsetzung

Die entwickelte Architektur wurde mit dem von JetBrains entwickelten Framework Compose Multiplatform (CMP)[10] entwickelt. Dieses ist eine Erweiterung von Kotlin Multiplatform (KMP)[9]. KMP ermöglicht es die Geschäftslogik einer App auf verschiedenen Targets (Plattformen) zu teilen. Mit CMP wird dann zusätzlich auch noch die Benutzeroberflächen auf allen Plattformen über eine Codebasis erstellt. Dabei können nach Bedarf plattformabhängige Implementierungen erstellt werden, theoretisch kann aber jede Plattform über nur eine Codebase angesprochen werden. CMP nutzt dafür hierfür eine Erweiterung von Jetpack Compose [11]. Da CMP noch recht neu ist, und zum Beispiel noch nicht alle Plattformen zum Start meines Projekt als stable galten, gibt es ein großes Angebot an Open Source-Lösungen für bestimmte Bereiche einer Anwendung. Eine Sammlung findet man im Repository **KMP Awesome**[12]. Für die Umsetzung meiner Anwendung war dies eine hilfreiche Ressource um mögliche Technologien zu finden.

5.1 Verwendete Technologien

Bei den Technologien musste stets auf die Abdeckung aller Plattformen geachtet werden. Oftmals gibt es in Compose Multiplatform bessere Community-Alternativen als die offiziell angebotenen Lösungen. Im Folgenden wird die Auswahl für mein Projekt kurz beschrieben, sowie auf verwendete Ressourcen verlinkt.

Benutzeroberfläche

Da es sich um ein CMP-Projekt handelt, wurde der Hauptteil der UI in Compose geschrieben. Dies ist von Haus aus stark an **Material Design** von Google [13] ausgerichtet. Theoretisch kann man das die UI-Komponenten aber vollkommen frei gestalten. Ich habe mich jedoch dafür entschieden, mich grob am Material Design zu orientieren und konnte dementsprechend hilfreiche Tools wie den **Material Design Builder** nutzen, der direkt zu Kotlin Code exportieren kann. Außerdem hat man direkt Zugriff auf die **Material Icons**. Während meines Projekts wurde viel an der neuen Navigationsbibliothek für Compose gearbeitet. Diese war allerdings zum Startzeitpunkt zumindest für Compose Multiplatform noch nicht stabil. Deswegen habe ich auf eine weitverbreitete Alternative gesetzt: **Voyager** [14], eine sehr robuste Navigationsbibliothek die deutlich stabiler und intuitiver wurde als die nunmehr veraltete Navigationsbibliothek von Compose selbst. Für die Implementierung der Kalender- und Stundenplanfunktionalität gab es ebenfalls eine gut anpassbare Bibliothek, die für Compose Multiplatform als weit verbreitet erschien: github.com/kozitonwose/Calendar. Für die Implementierung einer eigenen Karte inkl. Darstellung von lokalen GeoDaten und Nutzerstandort gab es leider nur ein aktiv weiterentwickeltes Projekt. **Maplibre Compose** [15] ist eine CMP-Wrapper um die MapLibre-SDK. Zum Zeitpunkt meines Projekts waren leider nur die mobilen Plattformen mit guten Funktionsumfang abgedeckt, Desktop und Web konnten keine eigenen GeoDaten in den Karten dar-



stellen und hatten zusätzlich einen deutlich erhöhten Einrichtungsaufwand. Als kleine visuelle Erweiterung entschied ich mich, leere Screens noch um Animated Vector Drawable (AVD) zu erweitern. Diese wirken meist ansprechender als nur ein einfacher Text, der darauf verweist, dass Inhalte nicht verfügbar sind. Für Compose Multiplatform gibt es allerdings keine Bibliothek, die das Lesen und Darstellen von AVDs ermöglicht. Stattdessen nutzte ich **Compottie**, welches ähnliche Funktionalität anbietet, aber stattdessen `.lottie`-Dateien nutzt. Das Erstellen von Lottie-Dateien ist beispielsweise über After Effects von Adobe oder den Lottie Creator von Lottie Files möglich.

Datenbeschaffung

Zur Anbindung von APIs gibt es das auf Kotlin basierende Client- und Server-Framework **Ktor** [16]. Mithilfe dessen man einfache, asynchrone Clients programmieren kann. Ktor ist auf allen Plattformen für Compose Multiplatform mit nur wenig Abhängigkeiten und ist sehr einfach integrierbar mit der **Serialisierungsbibliothek** von `kotlinx`. Für das Parsen von HTML-Inhalten bot sich durch die Interoperabilität von Kotlin mit Java die sehr etablierte Java-Bibliothek **jsoup** an.

Benutzereinstellungen und Persistenz

Für die persistente Datenhaltung gab es zwei Kandidaten: **SQLDelight**, eine Bibliothek, die zur Laufzeit aus SQL-Abfragen Kotlin-Code generiert oder die Datenbank **Room**, die mit KMP nutzbar ist [17]. Bei Bibliotheken basieren auf SQLite, Room bringt allerdings ein eigens Object-Relational Mapping (ORM) mit und schien aus Nutzerperspektive durch die stärkere Abstraktion angenehmer.

Die Benutzereinstellungen, also beispielsweise Einstellungen zum ausgewählten Campus, oder der Präferenz von Light oder Dark Mode wurden in einem einfachen Key-Value-Store mithilfe von **Multiplatform Settings** gespeichert und geladen.

5.2 Implementierung von Kernfunktionen

Im folgenden sollen zwei Kernfunktionen der App per Durchstich durch die Anwendungsarchitektur von oben (Nutzerschicht) bis zur Datenbeschaffung beschrieben werden.

Abfahrten und Anfahrten

Abfahrten können über den Abfahrtscreen abgerufen werden. Dabei handelt es sich um einen Voyager-Tab, mit einem dazugehörenden `ScreenModel` (Voyager-Variante von `ViewModel`). In diesem wird der Zustand des Screens gehalten: Welche Haltestellen stehen am aktuellen Campus zur Verfügung, welche Haltestelle ist ausgewählt, welche Abfahrten existieren oder ob gerade Abfahrten geladen werden. Außerdem bietet das `ScreenModel` die Anbindung an



die Datenbeschaffungsschicht an. Der Tab selbst besteht im wesentlichen aus einem Content-Composable mit Buttons für die Haltestellen und einer Liste von `DepartureEntry`-Composables. Die Buttons führen beim Klick eine Funktion des `ScreenModels` aus, welches wiederum die API anfragt um die Abfahrten zu besorgen. Dabei wird eine Lade-Flag gesetzt, damit der Nutzer informiert wird, dass Verbindungen gerade geladen werden. Die DTO-Rückmeldung der API wird dann ins Modell umgewandelt und in einer vom Tab beobachteten Liste abgespeichert. Dieser passt sich dementsprechend an und zeigt immer die aktuellen Anfahrten.

5.2.1 Kartenansicht des Campus

Die Kartenansicht wurden als eigener `Voyager`-Tab implementiert. Der Tab besteht im wesentlichen aus der plattformspezifischen Implementierung der `MapLibreMap` genannt `MapView`. Zusätzlich stellt dieser Tab applikationsweit die Möglichkeit bereit ein Gebäude zu fokussieren. Dem Tab wird ein `Voyager-ScreenModel` assoziiert, in dem das aktuelle Target (Fokus) der Karte gehalten und geändert werden kann.

Der `MapView` ist wiederum ein `Composable`, welches das `ScreenModel` beobachtet. Hier werden die Einstellungen der Karte gehalten (Gestensteuerung, Standardausrichtung, ...). Außerdem werden für die anzuzeigenden Campus-Gebäude Layer erstellt und Listener auf Nutzeraktionen gesetzt. Layer sind eine Möglichkeit auch zur Laufzeit weitere Daten auf der Karte anzuzeigen, die auf Nutzerinteraktionen über **Filter** reagieren. So hält das Karten-Composable beispielsweise das aktuell ausgewählte Feature (ein beliebiger Datenpunkt der Karte) als State und filtert so, welches Gebäude als `highlighted` dargestellt werden soll. Über den Feature-State wird außerdem eine Info-Karte mit dem Namen des ausgewählten Gebäudes eingeblendet.

5.3 Tests und Qualitätssicherung

Das automatisierte Testen wurde in diesem Projekt auf Unit Tests der Kernfunktionalitäten im plattformunabhängigen Code beschränkt. Dazu zählen Tests der API-Clients, Konvertierungstests zwischen DTO, Entity und Modell, der CRUD-Operationen auf der Datenbank und einige einfache Tests der UI.

Aufgrund des erhöhten Implementierungsaufwands für Android-Unit-Tests auf einem echten Emulator-Gerät wurden die meisten Tests nur ausführbar für das Desktop/JVM-Target geschrieben. Eine exemplarische Umsetzung mit Hilfe der Bibliothek `Roboelectric` wurde erstellt um zumindest die technische Machbarkeit nachzuweisen.

Tests der API-Clients

Zum Testen von Ktor Clients, auf denen die API-Clients im Projekt basierten stellt das Ktor-Framework die Möglichkeit bereit, die Clients mit einer `MockEngine` zu initialisieren. Diese Engine kann dann pro Test konfiguriert werden.



Tests von Konvertierungsmethoden

Die angebotenen Konvertierungsmethoden von DTO zu Domäne/Modell und Entity zu Modell und zurück wurden mit einfachen Test-Objekten überprüft. Außerdem wurden interne Parsing-Methoden mit einer größeren Menge an Testdaten getestet. Die zwei Testkategorien, auf die die Clients getestet wurden, war das korrekte Parsen von erhaltenen JSON-Strings zu DTOs und die korrekte Nutzung von Parametern in den jeweiligen HTTP-Requests.

Tests der Datenbank

Für Persistenztests musste ähnlich wie im Hauptprojekt für Room ein plattformspezifischer Datenbank-Builder bereitgestellt werden. Aufgrund der vorher beschriebenen Komplexität einen echten Android-Kontext in der Testumgebung zu erstellen wurde auch hier die Ausführbarkeit der Tests nur auf dem Desktop-Target ermöglicht.

Wie bereits einleitend erwähnt, wurden für alle Entities, die in der Datenbank enthalten sind, Tests der entsprechenden DAOs für die standardmäßigen CRUD-Operationen geschrieben. Hierzu wurden Testdaten angelegt, gelesen überschrieben und gelöscht und die erwarteten Ergebnisse überprüft:

Beispielsweise für eine Überschreibungsaktion, ob der Datensatz wirklich angepasst wurde, oder bei einer Löschaktion, ob der gelöschte Datensatz wirklich aus der Datenbank verschwunden ist.

Tests der Benutzeroberfläche

Die Tests der Benutzeroberfläche wurden mithilfe von Roboelectric [19] für die Android Plattform exemplarisch umgesetzt. Dabei wurde überprüft, ob Interaktionen mit der Einstellungsseite (Klick auf den Darkmode-Switch) wirklich den Appweiten State ändert. Mit relativ wenig Aufwand könnten so weitere UI-Tests implementiert werden.



6 Fazit

6.1 Zielerreichung und Ergebnisbewertung

Es wurde eine auf modernen Technologien basierende Multiplatform-Anwendung umgesetzt, die die betrachteten Use Cases ermöglicht und dabei eine erweiterbare und testbare Code-Basis nutzt. Einige Implementierungen basieren noch auf eher experimentellen Schnittstellen, die für die Wartbarkeit und zukünftige Betreibbarkeit herausfordernd erscheinen. Eine wahre Multiplatform-Erfahrung konnte aufgrund anfänglich fehlender Features des Frameworks im Rahmen dieses Projekts noch nicht erreicht werden. Trotzdem wurde der Grundstein gelegt, um zusätzliche Plattformen nachträglich hinzuzufügen.

6.2 Ausblick und Weiterentwicklung

Für dieses Projekt ergeben sich mehrere Möglichkeiten der Weiterentwicklung: Verbesserung der Code-Basis und Testabdeckung, Einrichten von neuen Plattformen und das Hinzufügen neuer Funktionalitäten. Dies ist ein Auszug der Verbesserungen, die ich im Laufe des Projekts gesammelt habe.

Verbesserungen in der Code-Basis

In der Codebasis sollte daran gearbeitet werden, dass die Benutzeroberfläche weiter abstrahiert wird. Komponenten sollte einheitlich gestaltet und zentraler verwaltet werden (Nicht mehr pro Tab) und eine Konfiguration für standardisierte Abstände eingeführt werden. Weiterhin sollte bei Entities, Modellen und DTOs in gemeinsame Superklasse abstrahiert werden. Dies würde das Handling über die gesamte App vereinfachen.

Zusätzlich sollten kleinere Änderungen umgesetzt werden. So sollte zum Beispiel auf die vermutlich auf die neue Navigation3-Bibliothek migriert werden. Außerdem sollte lokale Navigatoren eingebaut werden, da in Zukunft sicherlich auch Navigation innerhalb von Tabs notwendig wird.

Die lokalisierten Nachrichten sollten sauberer organisiert werden, um zu gewährleisten, dass sie auch bei einer wachsenden Architektur noch gut handhabbar sind.

Abschließend sollten GeoJSON und normales JSON für Gebäude zusammengeführt werden und eine einfache Möglichkeit zur Wartung der über JSON eingespielten Modelldaten entwickelt werden.



Verbesserung der Testabdeckung

Das Unit Testing sollte auf alle verbleibenden Komponenten der App erweitert und die bestehenden Tests robuster und weitgreifender gestaltet werden. Zusätzlich sollten Integrationstests erstellt werden, die das Zusammenspiel von verschiedenen Komponenten und Workflows simulieren und testen. Es sollte zusätzlich eine Lösung gesucht werden, mit der das UI-Testing komfortable für alle Plattformen gehandhabt werden kann.

Einrichten neuer Plattformen

In Zukunft sollten die beiden wichtigen Targets Web und iOS eingerichtet werden. Diese sind für ein Projekt wie eine CampusApp essentiell, da sie sinnvolle Verwendungszwecke für viele Studierenden sind (viele Studierende besitzen ein iPhone, und über das Web-Target ermöglicht man einen sehr niedrighschwelligen Zugriff auf die Anwendung). Aufgrund der verwendeten Abhängigkeiten sollten hier nur wenige plattformspezifische Lösungen notwendig sein.

Sinnvolle neue Funktionalitäten

Für die Orientierung am Campus wäre es sicherlich sinnvoll, auch die Orientierung innerhalb eines Gebäudes zu ermöglichen. Hierfür sollten also die Gebäudegrundrisse inkl. der verschiedenen Räume und ihrer Aufgaben eingepflegt werden. Sofern dies möglich ist, sollte dies auch innerhalb von MapLibre passieren, damit eine reibungslose Navigation zwischen Gebäuden auf dem Campus und Räumen in Gebäuden ermöglicht wird. Zudem sollten eine Routenführung des Nutzers von seinem aktuellen Standort zu einem anderen Standort eingebaut werden, damit man sich durch die App auch zu einem Gebäude lotsen lassen kann. Des weiteren wäre es sicherlich sinnvoll, die Campus-Karte um Fotos der Gebäude zu erweitern, um die Orientierung nicht nur durch die 3D-Modelle und ihre Position zu reduzieren.

Verschiedene weitere Hochschul-Systeme würden sicherlich erst bei einer wirklichen Umsetzung dieser App möglich. Dazu gehören unter Anderem die Anbindung des Kurs- und Prüfungssystems und der Lehrveranstaltungssysteme.

Sofern die Weiterentwicklung von MapLibre Compose es ermöglicht sollte die Programmierung der Kartenfunktionalitäten zukünftig im gemeinsamen Code für alle (zukünftigen) Plattformen erfolgen.



Literatur

- [1] Medieninformatik an der HSRM, hs-rm.de
- [2] uninow, uninow.de
- [3] Uni Augsburg Campus App, tha.de
- [4] RMV Open Data, opendata.rmv.de
- [5] Open Street Map, openstreetmap.org
- [6] Overpass Turbo, overpass-turbo.eu
- [7] Open-Meteo, open-meteo.com
- [8] Studiendenwerk Frankfurt, swffm.de
- [9] Kotlin Multiplatform, kotlinlang.org
- [10] Compose Multiplatform, kotlinlang.org
- [11] Jetpack Compose, developer.android.com
- [12] KMP Awesome, github.com
- [13] Material Design 3, m3.material.io
- [14] Voyager Navigation, yoager.adriel.cafe
- [15] MapLibre Compose, maplibre.org
- [16] Ktor, ktor.io
- [17] Room für KMP, developer.android.com
- [18] Testing Compose Multiplatform UI, kotlinlang.org
- [19] Roboelectric, roboelectric.org

Akronyme

AVD Animated Vector Drawable. 14

CMP Compose Multiplatform. 13

KMP Kotlin Multiplatform. 13

ORM Object-Relational Mapping. 14

